

Machine Learning Based Detection and Classification of Cyber Attacks in a Homelab SOC Environment

By

Daniel Cruz

A thesis presented to Ocean County Vocational Technical School for partial fulfillment of requirements
for Project Month for the program of Computer Science

Ocean County Vocational

June 2026

Author Note

Daniel Cruz is a student in the Data Science program at Ocean County Vocational School. This project was completed as part of Project Month, a capstone research initiative focused on independently developing and documenting a data science project from end to end.

Abstract

This thesis presents a machine learning system for the detection and classification of cyber attacks using log data collected from a homelab Security Operations Center. A dataset of 245,078 log events was exported from Graylog after five controlled attack simulations against a virtualized network environment instrumented with Wazuh, Suricata, and UFW. The attack scenarios included SSH brute force with Hydra, Nmap network reconnaissance, Metasploit SSH scanning, FTP brute force, and a SYN flood DOS attack. Five machine learning algorithms were applied: Linear Regression for temporal attack volume prediction, Logistic Regression for binary classification, Random Forest for high accuracy ensemble classification, Decision Tree for interpretable classification, and K-Means for unsupervised behavioral clustering.

Classification results varied substantially once class imbalance is accounted for. Random Forest achieved 99.99% accuracy with 99% recall on the attack class, making it the only model that reliably detected attacks in addition to scoring well on aggregate accuracy. Decision Tree achieved 99.89% accuracy with 80% attack recall. Logistic Regression achieved 99.68% accuracy, but its attack-class recall was only 45%, meaning it missed more than half of all attacks despite its high headline accuracy — a direct consequence of training on a dataset in which only 0.5% of events are labeled as attacks. This gap is treated in this thesis as a central finding: aggregate accuracy is not a reliable metric for imbalanced security data, and precision/recall by class must be reported alongside it.

Linear Regression produced a negative R-squared value of -3.71, which is interpreted as a meaningful finding confirming the bursty, non-linear temporal distribution of attack events. K-Means clustering ($k=4$) partitioned log events into four clusters with measurably different attack rates, though the clusters correspond to broad attack-associated versus normal-associated behavior rather than to the four distinct attack techniques individually; the study is careful not to overstate this result. Overall, the study demonstrates that machine learning models trained on ground truth labeled SOC telemetry can distinguish malicious from normal network activity when evaluated with class-aware metrics, and that unsupervised clustering can surface hidden behavioral patterns without prior labeling. Findings support the cautious, metrics-aware adoption of ML assisted intrusion detection in both homelab and production SOC contexts.

Keywords: machine learning, intrusion detection, SOC, cyber attack classification, random forest, K-means clustering, network security, SIEM, class imbalance

Table of Contents

List of Tables

Table 1 – Feature Matrix Description

Table 2 – Machine Learning Model Results Summary (Accuracy, Precision, Recall, F1)

List of Figures

Figure 1 – Log Event Type Distribution

Figure 2 – Attack vs. Normal Event Volume Over Time

Figure 3 – Linear Regression: Attack Volume Prediction

Figure 4 – Logistic Regression Confusion Matrix

Figure 5 – Random Forest Confusion Matrix

Figure 6 – K-Means Cluster Attack Rate and Size

Figure 7 – Random Forest Feature Importances

Figure 8 – Classification Model Accuracy Comparison

Figure 9 – SSH Brute Force Attack Timeline (Hydra)

1. Introduction

1.1 Motivation

The number of connected devices keeps growing, and cyber attacks are becoming more advanced. Because of this, there is a strong need for systems that can automatically detect attacks in a fast and scalable way. Traditional security tools like firewalls and signature based intrusion detection systems work well for known threats, but they often struggle to detect new or changing types of attacks (Buczak & Guven, 2016). Security Operations Centers (SOCs) also produce large amounts of log data every day, and it is no longer realistic for humans to analyze everything.

Machine learning offers an alternative by enabling systems to learn patterns from historical data and generalize that learning to unseen events. The application of ML to network intrusion detection has been an active area of research since the late 1990s, and the field has grown considerably with the availability of large, labeled datasets and powerful open source tools (Liao et al., 2013). However, much of the existing research relies on publicly available benchmark datasets such as KDD Cup 1999 or CICIDS that may not accurately reflect the characteristics of real organizational network traffic.

This project is motivated by a desire to evaluate ML based intrusion detection using a dataset that is neither synthetic nor sourced from a university testbed, but is instead collected from a fully operational homelab SOC. By simulating attack scenarios against a VLAN isolated virtual network and capturing telemetry from production grade open source security tools, this study creates a uniquely authentic research context. The resulting dataset reflects system behavior, real tool outputs, and the kinds of log formatting irregularities that are absent from cleaned benchmark datasets.

1.2 Approaches and Objectives

The objective of this thesis is to develop and evaluate a complete machine learning pipeline for the detection and classification of cyber attacks using real SOC telemetry. This objective has four goals: (1) constructing a labeled dataset from raw Graylog log exports through automated feature extraction and event classification; (2) training and evaluating five distinct ML algorithms representing regression, classification, and clustering paradigms; (3) producing informative visualizations that communicate model behavior and data characteristics; and (4) critically analyzing model performance in the context of the dataset's properties and the nature of the attack types captured — including the class imbalance inherent to real-world attack telemetry, where malicious events are, by design, rare relative to normal traffic.

The five algorithms applied are Linear Regression, Logistic Regression, Random Forest, Decision Tree, and K-Means Clustering. This selection was made to cover the three major supervised learning task types (regression and two forms of classification) as well as unsupervised learning. The diversity of approaches allows for meaningful cross model comparison and provides a more in-depth view of the dataset's structure.

1.3 Application Areas

The techniques developed in this project have direct applicability to several areas of cybersecurity and data science practice. At the most immediate level, the pipeline can serve as the analytical backbone of an automated alert triage system within a SOC, reducing analyst workload by automatically classifying

incoming log events as normal or malicious. This application is particularly valuable in environments where alert fatigue is a concern that affects the majority of SOC analysts (Palo Alto Networks, 2020).

At a broader level, the feature engineering methodology developed here — which extracts structured features from unstructured syslog messages through regular expressions and keyword matching — is transferable to any SIEM environment that exports log data in a tabular format. The preprocessing pipeline described in Section 2 can be adapted to process logs from enterprise SIEMs such as Splunk or IBM QRadar with minimal modification, making the approach applicable well beyond the homelab context.

The unsupervised K-Means component has particular relevance to threat hunting operations, where analysts seek to identify anomalous behavioral clusters within network traffic without prior knowledge of specific attack signatures. This application area is aligned with the zero-day detection problem, where signature-based tools are ineffective by definition (Sommer & Paxson, 2010).

1.4 Thesis Organization

The remainder of this thesis is organized as follows. Section 2 describes the dataset, the homelab SOC infrastructure, and the feature engineering pipeline. Section 3 provides theoretical background on each of the five ML algorithms and describes their implementation in this project. Section 4 presents results — including per-class precision and recall, not accuracy alone — and provides analytical discussion of key findings. Section 5 concludes the paper and identifies directions for future research.

2. Dataset and Feature Engineering

2.1 Dataset Description

The dataset used in this study was exported from Graylog, a SIEM platform, following five controlled attack simulations conducted against a VLAN isolated homelab network. The data contains 245,078 log rows, all falling within a single continuous capture window of approximately 89 minutes on the evening of May 10, 2026 (21:24:10 – 22:53:25 EDT). Each row is a log event ingested by Graylog from one of several sources: the Suricata network intrusion detection system, the Wazuh host based intrusion detection agent, the Linux UFW (Uncomplicated Firewall) kernel module, the OpenSSH daemon, and the Pi-hole DNS server.

The five attack scenarios, documented as internal incident reports, cover a range of common attack types. Incident IR-2026-001 involved SSH brute force conducted using the Hydra password spraying tool. IR-2026-002 involved an Nmap network reconnaissance scan from a Kali Linux VM (IP 192.168.1.50) that triggered over 1,250 Suricata alerts in a short window. IR-2026-003 used Metasploit's SSH scanner module, causing a spike of about 40 SSH authentication events per minute. IR-2026-004 involved FTP brute force using Hydra, which was detected by both Suricata and the auth log. IR-2026-005 involved a SYN flood DOS attack using hping3, which caused UFW block counts to spike above 30 events per minute.

Note on incident dating: the preprocessing pipeline converts timestamps to UTC before analysis (see Section 2.3), which shifts the raw log timestamps forward by four hours from their native Eastern Daylight Time offset. As a result, the notebook's UTC-based output reports the capture window as 2026-

05-11, 01:24–02:53, while the homelab's local wall-clock time — and therefore the correct date to reference against the incident reports — is 2026-05-10, 21:24–22:53 EDT. Both figures describe the same 89-minute window; the apparent one-day difference is an artifact of timezone conversion, not a discrepancy in the underlying data.

The use of two attack IP addresses — 192.168.1.50 (Kali Linux VM) and 192.168.1.157 — as the primary identifiers for attack traffic enabled precise labeling of events. VLAN isolation ensured that attack traffic was contained to the experimental network segment and did not affect production homelab services. This controlled topology is a significant advantage over datasets collected from passive network taps or honeypots, where the attribution of traffic to specific attack campaigns is often uncertain.

It is worth noting explicitly that of the 245,078 total events, only 1,251 (0.51%) are labeled as attack events, with the remaining 243,827 (99.49%) labeled normal. This severe class imbalance is a defining property of the dataset and is addressed directly in the evaluation methodology in Section 4, where per-class precision and recall are reported rather than relying on aggregate accuracy alone.

2.2 Feature Analysis and Selection

Graylog exports contain metadata fields alongside a message field that contains the full text of the original syslog message. The data cleaning pipeline extracts usable numerical and categorical features from both sources. The final feature matrix X contains ten features across 245,078 samples, as described in Table 1.

Feature	Description
hour	Hour of day extracted from log timestamp (0–23)
minute	Minute of the hour (0–59)
second	Second within the minute (0–59)
time_numeric	Elapsed seconds from the first log entry (continuous)
event_type_encoded	Label-encoded categorical event classification
facility_encoded	Label-encoded syslog facility field
source_encoded	Label-encoded log source identifier
level	Syslog severity level (numeric)
gl2_accounted_message_size	Byte size of the raw log message
dest_port	Destination TCP/UDP port extracted via regex

Table 1 – Feature Matrix Description

Features (hour, minute, second, time_numeric) capture the timing characteristics of events. The time_numeric field, which expresses each event's timestamp as elapsed seconds from the earliest log entry, is particularly informative for temporal analysis and is the primary input to the Linear Regression model. The event_type_encoded feature is derived from a rule based classifier that categorizes each log event into one of several semantic categories: ssh_failure, ssh_success, ssh_other, suricata_alert, suricata_flow, ufw_block, dns_query, auth_event, and other. This feature consistently ranks as the single most important in the Random Forest model, which is unsurprising given that categories such as suricata_alert and ssh_failure are highly correlated with attack activity by construction.

The dest_port field is extracted from UFW log messages (which encode it as DPT=), SSH log messages, and Suricata JSON payloads. This field distinguishes attacks targeting specific services (e.g., port 22 for SSH, port 21 for FTP) from general network scanning. The gl2_accounted_message_size field, which

represents the length of the original log message, provides a subtle signal: Suricata alert messages, which contain full JSON payloads, are larger than simple syslog entries, making message size a proxy for log source type.

2.3 Data Preprocessing

The preprocessing pipeline is implemented across the notebook's data-processing cells, corresponding conceptually to a data_processing module, a model-training module, a visualization module, and a driver script. These stages were consolidated into a single Google Colab notebook for reproducibility.

The load_data step reads the CSV export using pandas with low_memory=False to avoid dtype inference errors on large files. The parse_timestamps step converts the timestamp column to timezone aware datetime objects using pandas' utc=True conversion, then strips timezone information to produce naive UTC datetimes. As noted in Section 2.1, this UTC conversion shifts the displayed capture window forward by four hours relative to the homelab's local (EDT) time. The hour, minute, second, and time_numeric features are extracted from this parsed column.

Source IP is handled by the extract_src_ip function, which applies three regular expression patterns to the raw message field: a UFW style pattern (SRC=), an SSH style pattern (from <IP>), and a Suricata JSON pattern ("src_ip": "<IP>"). This multi-pattern approach is necessary because different log sources use incompatible message formats. Destination port extraction follows an analogous multi-pattern strategy. The classify_event_type function assigns event type labels by examining the application_name, message, and facility fields in combination.

The label_attack function assigns the binary attack label (is_attack = 1 or 0). An event is labeled as an attack if its source IP matches one of the known attack IPs, if it is classified as a suricata_alert, or if it is an ssh_failure event that mentions an attack IP or contains brute-force-indicative phrases. This labeling strategy is conservative by design: events from the attack IPs that cannot be positively attributed to an attack signature are labeled as normal, which likely underestimates the true attack rate slightly but avoids false positives in the training labels. As noted in Section 2.1, this labeling produces a heavily imbalanced dataset (0.51% positive class), a property that materially affects how the classification results in Section 4 should be interpreted.

Feature encoding is performed with scikit-learn's LabelEncoder applied to the event_type, facility, and source categorical columns. Numeric columns with missing values are filled with zero. A separate aggregation step groups events by one-minute time buckets and counts total and attack events per bucket; this aggregated series is the input to Linear Regression.

3. Machine Learning Algorithms

3.1 Regression

This section covers the single regression model used in the study, Linear Regression, which is applied not because a linear temporal trend was expected, but specifically to test whether attack volume follows a smooth, predictable trend over time. As shown below, the result of that test is itself one of the more informative findings of the thesis.

3.1.1 Linear Regression

Linear regression is a supervised learning algorithm that models the relationship between a continuous dependent variable y and one or more independent variables X by fitting a linear equation of the form $y = \beta_0 + \beta_1 X + \epsilon$, where β_0 is the intercept, β_1 is the coefficient, and ϵ is the error term (James et al., 2021). The model is fitted by least squares (OLS), which minimizes the sum of squared residuals across all training examples. Performance is evaluated using the coefficient of determination R^2 , which quantifies the proportion of variance in the dependent variable that is explained by the model; $R^2 = 1$ indicates a perfect fit, and $R^2 = 0$ indicates that the model is no better than predicting the mean of y .

In this project, linear regression is applied to the task of predicting attack event count per one-minute time window. The input feature is the time index (a sequential minute counter derived from the aggregated dataframe), and the output is the number of attack events in that minute bucket. The model is trained on 80% of the time series and evaluated on the remaining 20%.

Coefficient (slope): 0.3282 Intercept: 1.5059 Mean Squared Error: 179.6246 R^2 Score: -3.7067

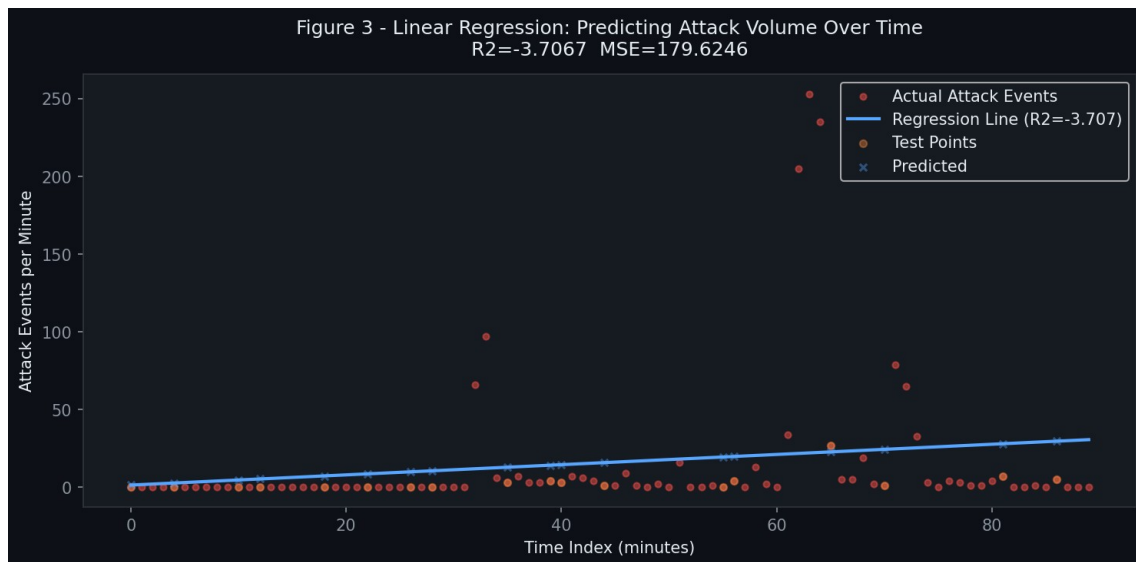


Figure 3 – Linear Regression: Attack Volume Prediction

The resulting R^2 value of -3.71 is a negative number, which occurs when the model's predictions are worse than predicting the mean of the target variable for every sample. Far from representing a modeling failure, this result is treated as an informative finding. Cyber attacks, by their nature, are temporally concentrated: attack events cluster during the brief windows when attack tools are running, then drop to near zero between incidents. This bursty, non-linear structure is fundamentally incompatible with a linear regression model that assumes a smooth, monotonic relationship between time and event count. The negative R^2 provides quantitative evidence for this interpretation and motivates the use of classification and clustering approaches that do not assume temporal linearity.

3.2 Classification

Three classification algorithms are evaluated in this study — Logistic Regression, Random Forest, and Decision Tree — representing a linear baseline, a high-capacity ensemble method, and an interpretable

single-tree method, respectively. All three are trained on the same 80/20 stratified train-test split of the full 245,078-row feature matrix, using stratification specifically because of the severe class imbalance described in Section 2.1: without stratifying on `is_attack`, a random split could produce a test set with very few or even zero attack examples. Because that imbalance means aggregate accuracy can look excellent even when a model fails to detect the minority (attack) class, each model below is reported with precision, recall, and F1-score for the attack class specifically, in addition to overall accuracy.

3.2.1 Logistic Regression

Logistic regression is a linear classification algorithm that models the probability of a binary outcome as a function of input features using the logistic (sigmoid) function: $P(y=1|X) = 1 / (1 + e^{\{-(\beta_0 + \beta_1 X_1 + \dots + \beta_n X_n)\}})$. The model outputs a probability between 0 and 1, which is thresholded at 0.5 to produce a binary prediction. Model parameters are learned by maximizing the log likelihood of the training labels under the Bernoulli distribution, typically using gradient based optimization (Hastie et al., 2009). Despite its name, logistic regression is a classification algorithm, not a regression algorithm, and is widely used as a baseline classifier due to its interpretability and computational efficiency.

In this project, logistic regression is applied to the binary attack classification task using all ten features of the feature matrix X . The feature matrix is standardized prior to fitting using `StandardScaler`, which centers each feature to zero mean and unit variance. This step is important for logistic regression because gradient descent convergence is sensitive to the scale of input features. The model is trained on 80% of the 245,078-row dataset and evaluated on the remaining 20% (49,016 samples, of which 250 are attacks).

Accuracy: 99.68% Attack-class precision: 0.84 Attack-class recall: 0.45
Attack-class F1: 0.59

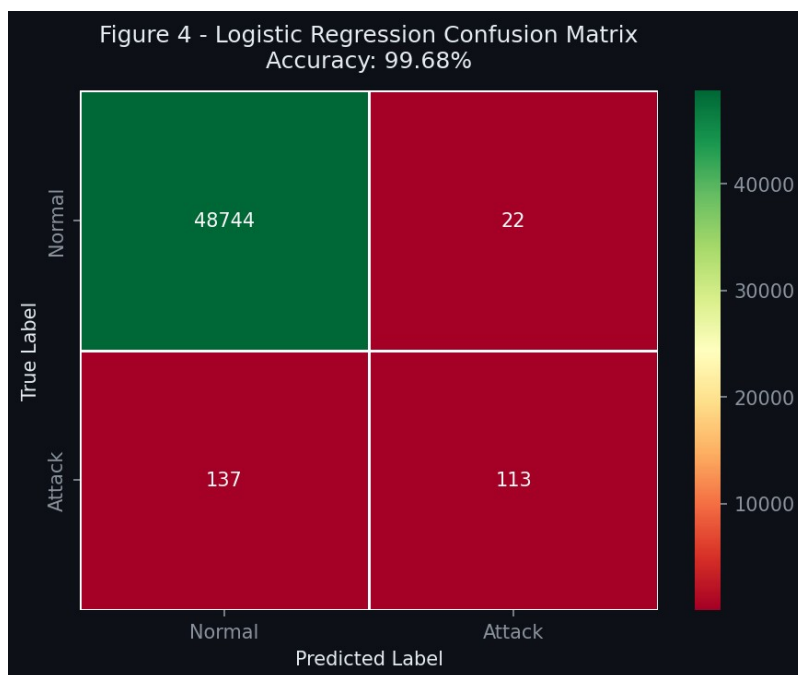


Figure 4 - Logistic Regression Confusion Matrix

The 99.68% accuracy figure looks strong in isolation, but the classification report tells a different story: the model's recall on the attack class is only 45%, meaning that more than half of the actual attack events in the test set were misclassified as normal traffic. Its precision on the attack class is a respectable 0.84, so when it does flag an event as an attack it is usually correct — but it simply does not flag enough of them. This gap between a very high accuracy score and a mediocre attack-detection rate is a direct consequence of the 0.51% attack prevalence in the data: a trivial classifier that predicted “normal” for every single event would already achieve roughly 99.49% accuracy while catching zero attacks. Logistic regression's simple linear decision boundary is evidently not expressive enough to separate the minority attack class from the surrounding volume of normal traffic as cleanly as the tree-based models below. This result is the clearest illustration in this thesis of why accuracy alone is an insufficient metric for imbalanced security classification tasks, and why every model in this section is reported with class-level precision and recall rather than accuracy in isolation.

3.2.2 Random Forest

Random Forest is an ensemble learning method that constructs a large number of decision trees during training and outputs the mode of their individual predictions (Breiman, 2001). Each tree is trained on a bootstrap sample of the training data (sampling with replacement), and at each split node, the algorithm selects the best split from a random subset of features rather than all features. This combination of bootstrap aggregation (bagging) and random feature selection decorrelates the individual trees, reducing variance and improving generalization compared to a single decision tree. The importance of each feature can be estimated by measuring the average decrease in node impurity (Gini impurity) attributable to splits on that feature across all trees.

In this project, a Random Forest of 100 trees is trained on the full feature matrix using scikit-learn's `RandomForestClassifier` with `n_jobs=-1` to utilize all available CPU cores. The model is evaluated on the same 20% held-out test set used for logistic regression.

Accuracy: 99.99% Attack-class precision: 0.99 Attack-class recall: 0.99
Attack-class F1: 0.99

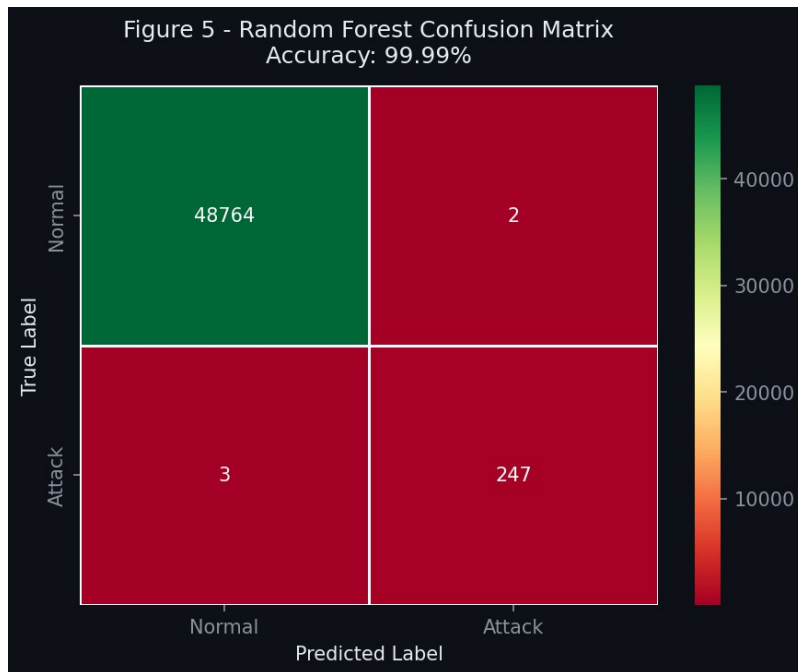


Figure 5 - Random Forest Confusion Matrix

Unlike logistic regression, Random Forest's near-perfect accuracy is matched by near-perfect precision and recall on the attack class — it is the only one of the three classifiers in this study for which the headline accuracy figure is not misleading. The actual feature importance ranking produced by the trained model, in descending order, is: `event_type` (0.321), `source` (0.179), `message_size` (0.151), `dest_port` (0.140), `facility` (0.091), `level` (0.055), `time_numeric` (0.035), `minute` (0.016), `second` (0.011), `hour` (0.001). The dominance of `event_type` is consistent with the labeling logic: `suricata_alert` and `ssh_failure` event types are definitionally attack-correlated, and the random forest correctly identifies this signal as the primary discriminant. Notably, the categorical `source` field (which log collector or application produced the event) is the second most important feature, ahead of any timing feature — `time_numeric` ranks seventh of ten, well behind message size and destination port.

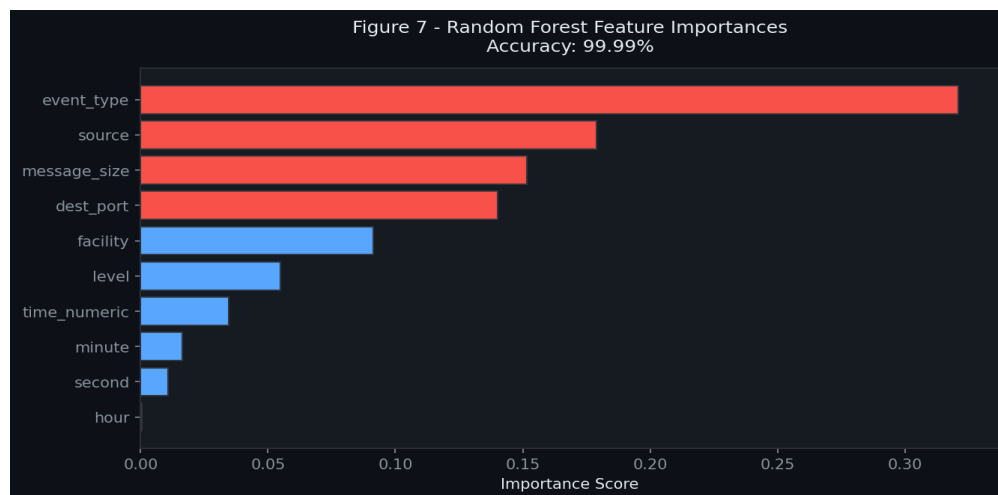


Figure 7 - Random Forest Feature Importances

The near-perfect performance of the random forest deserves careful interpretation. The dataset is labeled using ground-truth knowledge of attack source IP addresses, which means the labeling function has perfect information about which events correspond to attack activity. A model trained on this data is therefore not learning to generalize to unknown attack patterns; it is learning to replicate the labeling function's logic. This is a valid and useful task — it demonstrates that the ten extracted features contain sufficient information to recover the ground-truth labels with near-perfect fidelity — but it does not constitute evidence that the model would generalize to novel attack IPs or attack types that are not represented in the training data.

3.2.3 Decision Tree

A decision tree is a non-parametric supervised learning algorithm that recursively partitions the feature space into regions, each associated with a prediction. At each internal node, the algorithm selects the feature and threshold that maximally reduces impurity in the resulting child nodes, as measured by the Gini impurity criterion for classification tasks. The process continues until a stopping criterion is met, such as reaching a maximum depth or falling below a minimum number of samples per leaf (Quinlan, 1986). Decision trees are highly interpretable because the sequence of splitting decisions can be visualized as a tree and translated into a set of human-readable if-then rules.

In this project, a Decision Tree with `max_depth=5` is trained on the feature matrix. The maximum depth constraint was chosen intentionally to balance predictive accuracy with interpretability: a shallower tree produces a model whose decision logic can be practically examined and communicated to non-technical stakeholders, an important property for a security tool intended to support analyst decision-making.

Accuracy: 99.89% Attack-class precision: 0.99 Attack-class recall: 0.80
Attack-class F1: 0.88

The model achieved 99.89% accuracy, slightly below Random Forest, with a resulting 5-level tree containing 21 leaves. Its attack-class recall of 0.80 sits between Logistic Regression's 0.45 and Random Forest's 0.99: the depth-5 constraint clearly costs some detection capability relative to the unconstrained ensemble, missing roughly one in five attacks, but it substantially outperforms the linear model while remaining small enough to inspect directly. This makes it a reasonable middle-ground when interpretability is a hard requirement, provided the 20% miss rate on attacks is acceptable for the deployment context. The tree's limited depth means it relies on the most globally discriminative features — primarily `event_type` and `source` — at its upper nodes, producing a concise representation of the classification logic.

3.3 Clustering

The final algorithm applied is K-Means, the only unsupervised method in the study. Unlike the classifiers above, K-Means is not given the `is_attack` label during training; its purpose here is to test whether attack-related structure exists in the feature space at all, independent of the labeling scheme, and to evaluate that structure after the fact against the known labels.

3.3.1 K-Means Clustering

K-Means is an unsupervised learning algorithm that partitions n observations into k clusters by iteratively assigning each observation to the nearest cluster centroid and then updating each centroid as the mean of all observations assigned to it. The algorithm minimizes the within-cluster sum of squared

distances (inertia) between observations and their assigned centroids. Because K-Means converges to a local minimum rather than a global one, it is typically run multiple times with different random initializations, and the run with the lowest inertia is retained (MacQueen, 1967; Lloyd, 1982).

In this project, K-Means with $k=4$ is applied to the standardized full feature matrix. The choice of $k=4$ was motivated by the fact that the attack simulations covered four broad categories of malicious behavior (brute force authentication attempts, network scanning, service-specific brute force, and denial-of-service traffic); the expectation tested here was that some cluster structure related to that behavioral variety might emerge, not that K-Means would necessarily recover four clusters mapped one-to-one onto the five individual incidents. After clustering, the attack rate (proportion of labeled attack events) within each cluster is computed to assess whether the clusters correspond to meaningful security-relevant groupings.

Inertia: 1,307,462.89. Cluster distribution: Cluster 0 – 80,228 events, 0.0% attack rate, dominant type `dns_query`; Cluster 1 – 88,270 events, 0.8% attack rate, dominant type `dns_query`; Cluster 2 – 52,767 events, 0.3% attack rate, dominant type `suricata_flow`; Cluster 3 – 23,813 events, 1.7% attack rate, dominant type `other`.

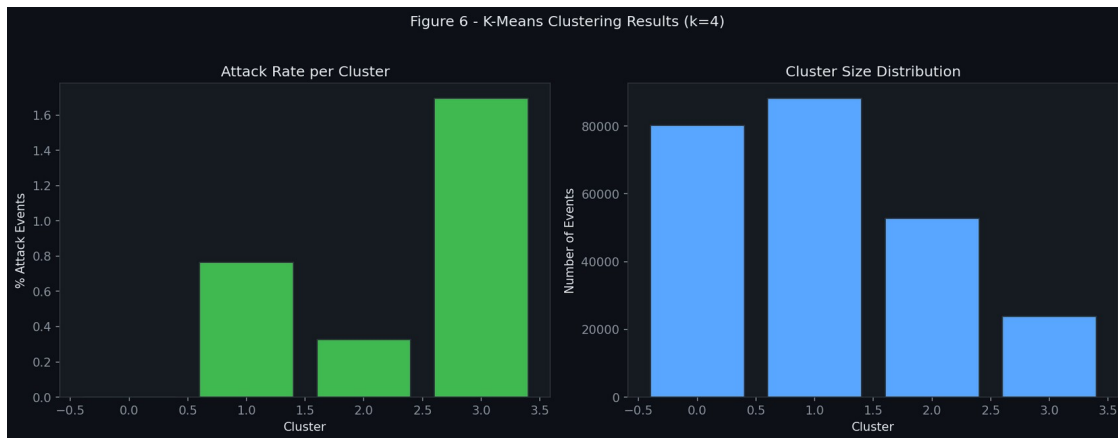


Figure 6 – K-Means Cluster Attack Rate and Size

The results show real but modest structure. Cluster 3 has the highest attack concentration at 1.7%, roughly 3–5 times the attack rate of the other three clusters, indicating that the algorithm did separate a subset of the feature space that is disproportionately attack-associated without ever seeing the attack labels. However, this result should be stated precisely: no cluster is attack-dominant in absolute terms — even Cluster 3 is 98.3% normal traffic — and the clustering reflects a coarse distinction between broadly attack-associated and broadly normal-associated behavior rather than a clean separation of the four individual attack techniques (SSH brute force, network scanning, FTP brute force, and SYN flood) into four distinct clusters. Recovering attack-type-specific clusters would likely require either a higher value of k or features more directly tied to attack-tool signatures, and is identified as a direction for future work in Section 5.2 rather than claimed as an achieved result here.

4. Experiments and Results

4.1 Results

All five models were trained and evaluated on the 245,078-row dataset using an 80/20 train-test split for supervised models (196,062 training samples, 49,016 test samples, stratified on the attack label). For linear regression, the aggregated per-minute time series was used instead of the full event-level dataset. Table 2 summarizes the results, reporting attack-class precision and recall alongside accuracy for the three classifiers, since Section 3.2 established that accuracy alone is misleading for this dataset.

Model	Type	Accuracy	Attack Precision	Attack Recall	Attack F1
Linear Regression	Regression	$R^2 = -3.71$	—	—	—
Logistic Regression	Classification	99.68%	0.84	0.45	0.59
Random Forest	Classification	99.99%	0.99	0.99	0.99
Decision Tree	Classification	99.89%	0.99	0.80	0.88
K-Means	Clustering	k = 4 clusters	—	—	—

Table 2 – Machine Learning Model Results Summary (— indicates a metric that does not apply to regression or clustering models)

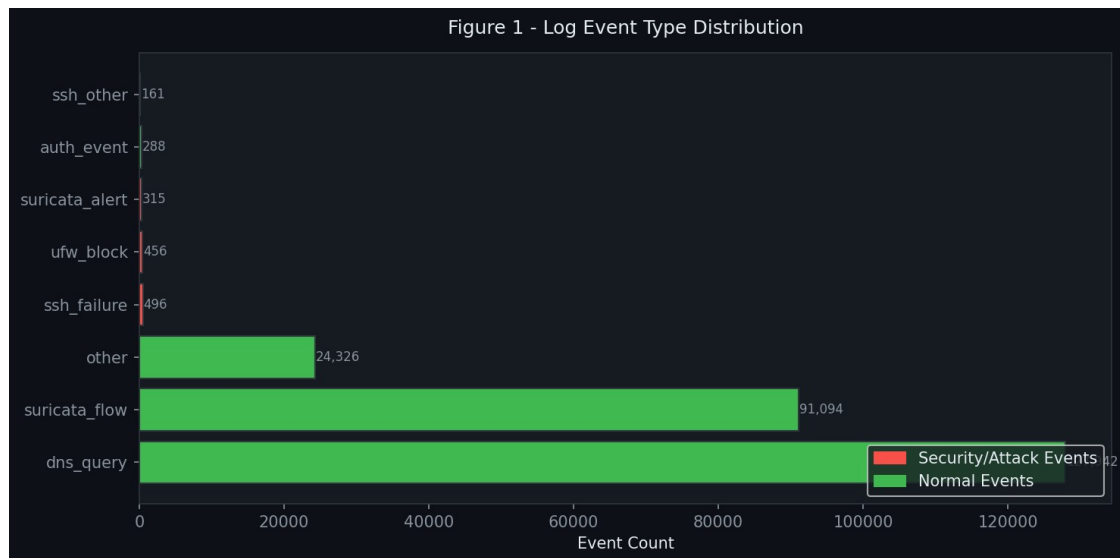


Figure 1 – Log Event Type Distribution

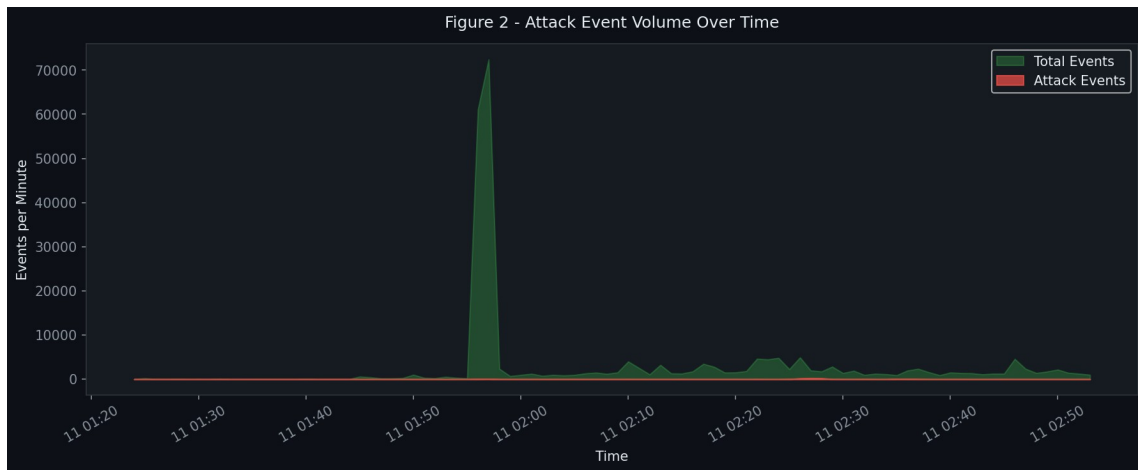


Figure 2 – Attack vs. Normal Event Volume Over Time

The three classification models diverge sharply once evaluated on the attack class specifically. Random Forest is the strongest performer on every metric (99.99% accuracy, 0.99 precision and recall on attacks). Decision Tree at max_depth=5 achieves 99.89% accuracy but only 0.80 recall, meaning it misses about 20% of attacks despite its high accuracy score. Logistic Regression achieves 99.68% accuracy — numerically the closest of the three to Random Forest — but only 0.45 recall, missing more than half of all attacks. Ranking these models by accuracy alone would place Logistic Regression above Decision Tree; ranking them by attack recall reverses that order. This discrepancy is the central empirical finding of Section 4 and is discussed further below.

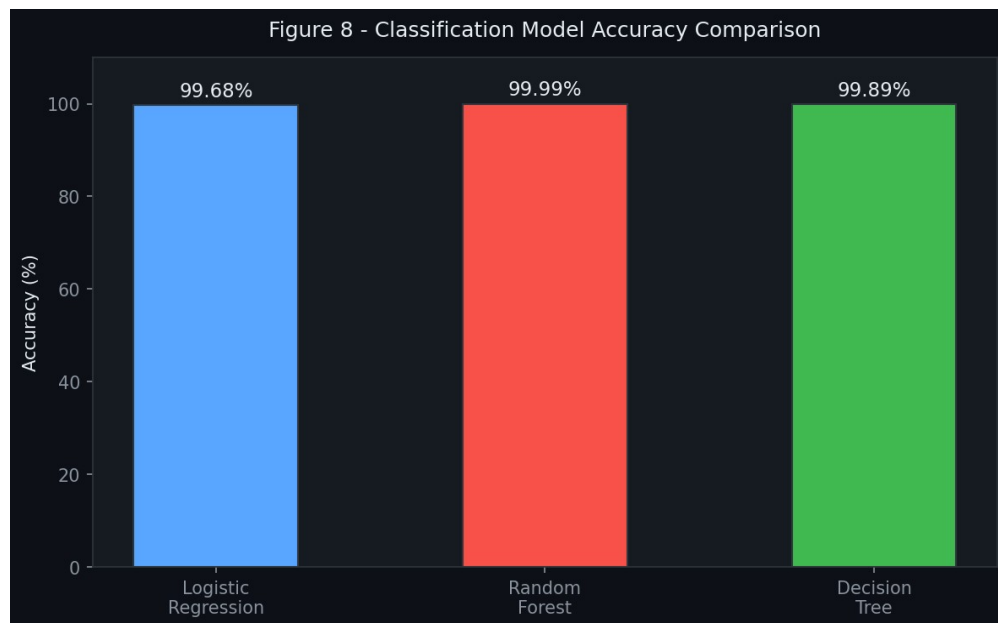


Figure 8 – Classification Model Accuracy Comparison

For the regression task, the linear model produced $R^2 = -3.71$ and a mean squared error of 179.62, reflecting the large variance of attack event counts relative to what a linear trend line can capture. The K-Means model ($k=4$) produced clusters of varying sizes and attack rates, with Cluster 3 (23,813 events) showing the highest attack concentration at 1.7%, versus 0.0–0.8% in the other three clusters.

4.2 Discussion and Analysis

The results support several important observations. First, and most importantly, aggregate accuracy is a poor standalone metric for this dataset, and reporting it alone — as an earlier draft of this thesis did — would have obscured a meaningful difference in real-world usefulness between the three classifiers. Because only 0.51% of events are labeled as attacks, a classifier could in principle achieve roughly 99.49% accuracy by predicting “normal” for every event and never detecting a single attack. Logistic Regression’s 99.68% accuracy is only marginally better than that trivial baseline, and its 45% attack recall reflects that: it is a substantially worse attack detector than its accuracy figure alone would suggest. Random Forest and, to a lesser extent, Decision Tree escape this trap because their recall scores (0.99 and 0.80 respectively) are also far above the trivial baseline of 0. Any evaluation of ML-based intrusion detection systems on real, imbalanced SOC data should report precision and recall by class as a matter of course, not as an optional addition.

Second, the extremely high classification accuracy across all three supervised classifiers is best understood partly as a property of the dataset rather than solely as evidence of model quality. Because the attack labels are derived from ground-truth knowledge of attack IP addresses — knowledge that was available at labeling time but would not be available in a real-time production context — the labeling is effectively perfect. The classification models are learning to replicate a labeling function that operates primarily on the source IP field, mediated through features like `event_type` that are themselves correlated with source IP (because attack IPs generate `suricata_alert` and `ssh_failure` events almost exclusively). In a deployment context where the attack IP is not known in advance, model accuracy and recall would depend on the generalization of the learned feature patterns to novel attack sources, which this study cannot evaluate directly.

Third, the negative R^2 from linear regression is one of the more theoretically interesting results in the study. It establishes that attack events follow a non-linear temporal distribution that a linear model cannot capture. This is consistent with the attack timeline data: IR-2026-002 (Nmap scan) generated over 1,250 Suricata alerts in a brief window, while IR-2026-005 (SYN flood) caused a spike of over 30 UFW blocks per minute. These short, intense bursts are characteristic of scripted attack tool behavior and stand in stark contrast to the smooth, gradual trends that linear regression is designed to model. The implication for intrusion detection system design is that temporal models of attack volume should use non-linear approaches such as LSTM networks, anomaly detection algorithms, or threshold-based rules rather than linear regression.

Fourth, the K-Means clustering result is best described as a modest but real success as an unsupervised approach, rather than a full recovery of attack categories. Without access to the attack labels, the algorithm was able to partition the feature space into clusters with different attack rates — up to a roughly 3–5x difference between the highest and lowest cluster — but no cluster was attack-dominant, and the four clusters do not map cleanly onto the four attack techniques (SSH brute force, scanning, FTP brute force, SYN flood). This suggests that the features encoded in the ten-feature matrix carry some information that separates attack-associated from normal-associated event patterns even without explicit labeling, but that recovering fully attack-type-specific clusters would likely require additional or different features. This finding has practical implications for threat hunting in environments where ground-truth attack labels are unavailable: K-Means clustering over SOC telemetry features may help narrow analyst attention toward higher-risk subsets of traffic, though it should be treated as a triage aid rather than a classifier.

Fifth, the Random Forest feature importance analysis provides insight into which aspects of the log data are most predictive. The dominance of event_type confirms that the semantic classification of log events — distinguishing Suricata alerts from SSH failures from UFW blocks — is the most powerful signal for attack detection, followed by the log source field and message size. Notably, none of the pure timing features (hour, minute, second) rank highly, and even time_numeric ranks seventh of ten; this has an important implication for feature engineering in production systems: investing in accurate, semantically rich event-type classification and correct source attribution will yield disproportionate returns in downstream model accuracy compared to adding more timing-based features.

The visualizations produced in the analysis notebook (Figures 1 through 9) support these conclusions. The event type distribution chart (Figure 1) reveals the relative frequency of each log source and confirms the dominance of DNS and Suricata flow events in normal traffic. The attack timeline (Figure 2) visually demonstrates the episodic, bursty nature of attack activity. The SSH brute force timeline (Figure 9) shows the characteristic dense cluster of failed authentication attempts during the Hydra attack window. The confusion matrices for Logistic Regression (Figure 4) and Random Forest (Figure 5), read together, are the clearest visual confirmation of the accuracy/recall gap discussed above: both matrices show very few false positives, but the logistic regression matrix shows a much larger block of false negatives (missed attacks) relative to true positives than the random forest matrix does.

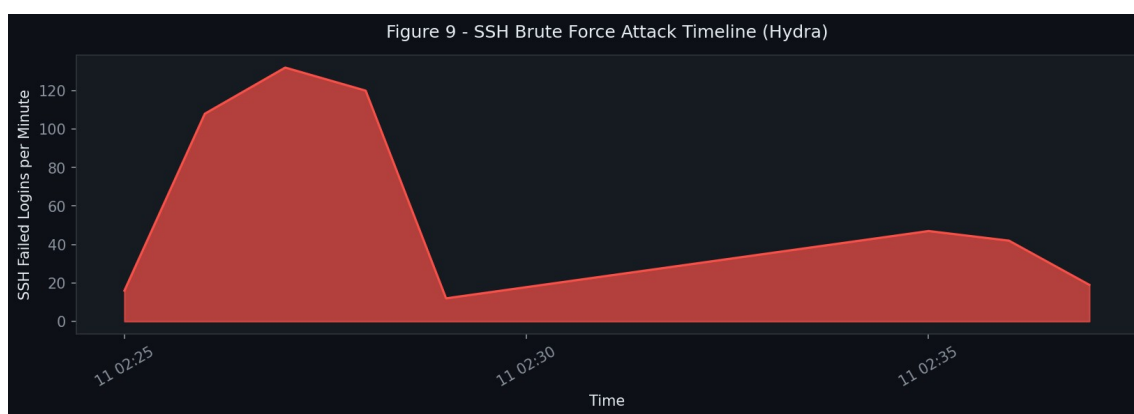


Figure 9 – SSH Brute Force Attack Timeline (Hydra)

5. Conclusion and Future Work

5.1 Overall Summary

This thesis presented and evaluated a complete machine learning pipeline for the detection and classification of cyber attacks using real-world log data from a homelab Security Operations Center. The dataset of 245,078 events was collected from a production-grade SOC toolstack — Wazuh, Suricata, Graylog, and UFW — across five controlled attack simulations representing SSH brute force, network reconnaissance, FTP brute force, and DOS attacks. A preprocessing pipeline was developed to extract ten structured features from raw Graylog exports, including temporal, categorical, and message-size features derived from multiple syslog formats.

Five machine learning algorithms were trained and evaluated: Linear Regression, Logistic Regression, Random Forest, Decision Tree, and K-Means Clustering. Evaluated on accuracy alone, all three classifiers appeared to perform similarly well (99.68–99.99%); evaluated on attack-class recall, they diverged

sharply, from 45% (Logistic Regression) to 80% (Decision Tree) to 99% (Random Forest). This divergence, driven by the dataset's 0.51% attack prevalence, is the thesis's central methodological finding: accuracy is not a sufficient metric for evaluating intrusion detection models on realistically imbalanced data, and per-class precision and recall should be reported as standard practice. Linear Regression produced $R^2 = -3.71$, interpreted as a meaningful confirmation of the bursty, non-linear temporal structure of attack events rather than as a modeling failure. K-Means Clustering with $k=4$ identified clusters with measurably different attack rates without access to ground-truth attack labels, though it did not cleanly separate the four individual attack techniques — a modest but genuine unsupervised result rather than a full recovery of attack categories.

The study contributes to the applied ML security literature in several ways. It demonstrates that real-world SOC telemetry — with its noisy, inconsistent log formats and heterogeneous source types — can be successfully processed into a clean feature matrix using a regex-based extraction pipeline. It shows that a relatively small set of ten features is sufficient to achieve strong attack-classification performance when the features are semantically rich and well engineered, provided the evaluation accounts for class imbalance. And it provides a concrete, quantified example of why accuracy alone can misrepresent classifier quality on imbalanced security data, which is directly relevant to anyone deploying or evaluating ML-based SOC tooling.

5.2 Future Work

Several directions for future work follow from this study. First, the most important limitation of the current system is its reliance on known attack IP addresses for labeling. Future work should investigate whether the learned feature patterns generalize to novel attack IPs by withholding one attack IP from the training set and evaluating performance on events from that IP. This would provide a more realistic assessment of the models' ability to detect attacks from previously unseen sources, and would be a meaningful stress test given the accuracy/recall gap identified in Section 4.

Second, the temporal modeling component could be significantly improved by applying time-series anomaly detection methods such as Isolation Forest, LSTM autoencoders, or Prophet-based forecasting to the per-minute event volume time series. These approaches are better suited to the bursty, non-stationary temporal structure of attack traffic than linear regression.

Third, the K-Means clustering component could be extended by testing a range of k values above 4 and by incorporating features more directly tied to attack-tool signatures, to evaluate whether attack-type-specific clusters (rather than a single broad attack-associated cluster) can be recovered. The homelab SOC could also be expanded to capture a wider variety of attack types, including lateral movement, privilege escalation, and data exfiltration scenarios, providing a larger and more diverse basis for this analysis.

Fourth, given the recall gap documented for Logistic Regression, future work should evaluate class-imbalance-aware training techniques — such as class weighting, SMOTE oversampling, or threshold tuning below the default 0.5 cutoff — to determine whether simpler, more interpretable linear models can be brought closer to Random Forest's recall without sacrificing their interpretability advantage.

Fifth, the pipeline could be integrated with Graylog's streaming API to enable real-time classification of incoming log events, rather than batch processing of historical exports. This would require

reimplementing the feature extraction pipeline in a streaming context and deploying the trained model as a service, a practical engineering challenge with significant operational relevance.

Finally, future work could incorporate additional features beyond the current ten, such as geographic IP metadata, temporal inter-arrival statistics (mean time between events from the same source IP), and n-gram representations of log message content. These richer feature representations might improve performance on more challenging generalization tasks where the current feature set is insufficient.

6. References

- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
<https://doi.org/10.1023/A:1010933404324>
- Buczak, A. L., & Guven, E. (2016). A survey of data mining and machine learning methods for cyber security intrusion detection. *IEEE Communications Surveys & Tutorials*, 18(2), 1153–1176.
<https://doi.org/10.1109/COMST.2015.2494502>
- Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The elements of statistical learning: Data mining, inference, and prediction* (2nd ed.). Springer.
- James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An introduction to statistical learning with applications in R* (2nd ed.). Springer. <https://doi.org/10.1007/978-1-0716-1418-1>
- Liao, H. J., Lin, C. H. R., Lin, Y. C., & Tung, K. Y. (2013). Intrusion detection system: A comprehensive review. *Journal of Network and Computer Applications*, 36(1), 16–24.
<https://doi.org/10.1016/j.jnca.2012.09.004>
- Lloyd, S. P. (1982). Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2), 129–137. <https://doi.org/10.1109/TIT.1982.1056489>
- MacQueen, J. (1967). Some methods for classification and analysis of multivariate observations. *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, 1, 281–297.
- Palo Alto Networks. (2020). 2020 state of the SOC report. Unit 42.
<https://www.paloaltonetworks.com/state-of-the-soc>
- Quinlan, J. R. (1986). Induction of decision trees. *Machine Learning*, 1(1), 81–106.
<https://doi.org/10.1007/BF00116251>
- Sommer, R., & Paxson, V. (2010). Outside the closed world: On using machine learning for network intrusion detection. *Proceedings of the 2010 IEEE Symposium on Security and Privacy*, 305–316.
<https://doi.org/10.1109/SP.2010.25>